

REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

MINISTRE DE L'ENSEIGNEMENT
SUPERIEUR

ECOLE NATIONALE SUPERIEURE
POLYTECHNIQUE DE DOUALA



REPUBLIC OF CAMEROON

Peace – Work – Fatherland

MINISTRY OF HIGHER
EDUCATION

NATIONAL ADVANCED SCHOOL OF
ENGINEERING DOUALA



UE : PROGRAMMATION SYSTEME

**THEME : RustShell : un shell POSIX
minimaliste**

Filière : Génie Logiciel 4

MEMBRES DU GROUPE N°3

N°	NOMS	MATRICULES
1	AKOUDJOU MBA NOELLY CHRISTELA	22G00007
2	ASSOMO MARIELLE CASSANDRA DE PARIS	22G00016
3	EYOU M EBELLE JACQUES HERVE	22G00127
4	FOHOM GAINGNE MERVEILLES DIANE	22G00136
5	GUEMBU SIME GUY VALAIRE	22G00460
6	GUEMOGNE GERALDINE ESTHER	22G00153
7	NKAMOLOUM TENE DAMIEN ANTHONY	22G00328
8	PETNOU HONLUE FREDERIC ARMEL	22G00362
9	TAWE KUITCHE MARIUS PEGUY	22G00383
10	TIDO KEUBOU LEAL BRAEL	22G00393

Sous la supervision de : **Dr. MIMBEU**

Année Académique : 2025-2026

Résumé exécutif

RustShell est un shell interactif de type POSIX développé entièrement en Rust, sans recourir à des bibliothèques shell existantes. Ce projet démontre une maîtrise complète de la programmation système bas niveau : appels système POSIX (fork, execve, pipe, waitpid), contrôle du terminal en mode raw, gestion des signaux, et implémentation d'un interpréteur de commandes complet.

Le projet couvre cinq domaines fondamentaux de l'informatique système : l'analyse syntaxique (lexer/parser), l'exécution de processus, la communication inter-processus (pipes), la gestion des jobs, et l'interaction homme-machine via le terminal. À ces fondations s'ajoutent des fonctionnalités avancées — expansion de wildcards (glob), substitution de commande, prompt dynamique intégrant la branche Git courante — qui élèvent le projet au niveau d'un outil véritablement utilisable au quotidien.

La grande majorité des projets étudiants portent sur des applications web ou des algorithmes. RustShell se distingue en attaquant un problème de programmation système fondamental — écrire un shell — dans un langage moderne réputé difficile. Le résultat est un binaire de 531 Ko, entièrement autonome, qui peut remplacer bash pour des tâches courantes.

Table des matières

Résumé exécutif	1
1. Contexte et motivation	4
1.1 Pourquoi écrire un shell en 2025 ?	4
1.2 Pourquoi Rust plutôt que C ?	4
1.3 Pertinence dans l'écosystème actuel	4
2. Architecture technique	5
2.1 Vue d'ensemble	5
2.2 Pipeline de traitement d'une commande	5
2.3 Gestion des pipes Unix	5
2.4 Mode raw terminal et édition de ligne	6
3. Fonctionnalités implémentées	7
3.1 Récapitulatif des cinq parties	7
3.2 Fonctionnalités avancées (bonus)	7
4. Qualité du code et ingénierie	8
4.1 Métriques	8
4.2 Gestion des erreurs	8
4.3 Sécurité mémoire	8
5. Pertinence et cas d'adoption	9
5.1 Qu'est-ce qui rend RustShell adoptable ?	9
5.2 Scénarios d'usage concrets	9
5.3 Feuille de route : vers un shell production-ready	9
6. Apprentissages et compétences développées	11
6.1 Programmation système	11
6.2 Théorie des compilateurs appliquée	11
6.3 Rust avancé	11
7. Conclusion	12
Annexe — Commandes de référence	13

1. Contexte et motivation

1.1 Pourquoi écrire un shell en 2025 ?

Chaque fois qu'un développeur tape une commande dans son terminal, il interagit avec un programme remarquablement complexe. Le shell orchestre silencieusement des dizaines d'appels système : il crée des processus, configure des descripteurs de fichiers, gère des signaux, maintient un environnement d'exécution. Comprendre ce mécanisme de l'intérieur est l'un des exercices de programmation système les plus formateurs qui soit.

Les shells existants — bash, zsh, fish — sont des projets matures de plusieurs centaines de milliers de lignes, écrits en C dans les années 1970-1990. Leur code est difficile à lire, leur architecture historiquement chargée. Écrire son propre shell depuis zéro force à résoudre chaque problème sans filet : pas de bibliothèque readline pour le terminal, pas de parser shell générique, pas d'abstraction des appels système.

1.2 Pourquoi Rust plutôt que C ?

Le choix de Rust pour un projet système n'est pas anodin. Ce langage, développé par Mozilla Research et adopté massivement par l'industrie depuis 2016, apporte des garanties que le C ne peut offrir :

Problème en C	Solution apportée par Rust
Buffer overflow sur les fds non fermés	Le compilateur garantit la fermeture via le trait Drop
Use-after-free sur les structures de processus	Le borrow checker interdit les références invalidées
Conditions de course sur la table des jobs	Le système de types rend les data races impossibles à la compilation
Fuites mémoire sur les chemins d'erreur	La gestion des erreurs par Result<T,E> force le traitement explicite

Le résultat concret : RustShell a été développé sans aucun crash lié à la mémoire, sans utiliser Valgrind, sans passer des heures à déboguer des segfaults — des problèmes chroniques dans tout projet système C.

1.3 Pertinence dans l'écosystème actuel

Rust est désormais le second langage officiel du noyau Linux (depuis 2022). Microsoft, Google, Amazon et la NSA recommandent son adoption pour le code système. Le marché de l'emploi confirme cette tendance : les offres d'emploi mentionnant Rust ont augmenté de 400% entre 2020 et 2024. Maîtriser la programmation système en Rust est une compétence rare et recherchée. Selon Stack Overflow Developer Survey 2024, Rust est le langage "le plus aimé" par les développeurs pour la 9ème année consécutive. Les salaires des développeurs Rust système se situent dans le top 5% des rémunérations en ingénierie logicielle.

2. Architecture technique

2.1 Vue d'ensemble

RustShell est structuré en huit modules indépendants, chacun avec une responsabilité unique et des interfaces claires. Cette architecture modulaire facilite la maintenance, les tests, et l'extension future.

Module	Responsabilité	Interfaces clés
lexer/	Tokenisation de la ligne de commande	tokenize(input) → Vec<Token>
parser/	Construction de l'AST de commandes	parse(tokens) → CommandList
executor/	Fork/exec, builtins, expansion	execute_list(list, ctx) → i32
jobs/	Table des jobs, signaux POSIX	JobTable, builtin_fg/bg()
history/	Historique persistant + autocomplétion	History::add/prev/next()
glob/	Expansion wildcards * ? [...]	expand_args(argv) → Vec<String>
prompt/	Prompt dynamique Git + couleurs	build_prompt(...) → String
terminal/	Mode raw, Ctrl+R, édition de ligne	readline(prompt, history)

2.2 Pipeline de traitement d'une commande

Chaque ligne saisie par l'utilisateur traverse un pipeline de transformation en quatre étapes :

Étape 1 - Lexing : la chaîne brute est découpée en tokens typés (Word, Pipe, RedirectOut, Background...). Les guillemets simples et doubles sont gérés avec des sémantiques différentes, le backslash active l'échappement caractère par caractère.

Étape 2 - Parsing : les tokens forment une hiérarchie CommandList → PipelineItem → Pipeline → Command. Chaque commande embarque ses arguments et ses redirections. Les connecteurs (;, &&, ||) définissent la logique d'enchaînement.

Étape 3 - Expansion : l'executor parcourt les arguments pour substituer les variables (\$VAR, \${VAR}, \$?), exécuter les substitutions de commande \$(cmd), puis appliquer l'expansion glob (*.rs → main.rs lib.rs...).

Étape 4 - Exécution : les builtins sont traités directement dans le processus shell. Les commandes externes passent par fork/exec avec configuration des pipes et redirections via dup2().

2.3 Gestion des pipes Unix

L'implémentation des pipes illustre parfaitement la complexité cachée derrière l'opérateur | :

```
ls src/ | grep .rs | wc -l
```

Pour un pipeline de N commandes, RustShell crée N-1 pipes avec libc::pipe(), distribue les extrémités en/lecture aux processus dans le bon ordre, lance les N processus en parallèle, puis

ferme toutes les extrémités dans le processus parent. La fermeture correcte des descripteurs de fichiers est critique : un fd non fermé maintient le pipe ouvert et bloque indéfiniment le processus en lecture.

Descripteurs de fichiers

Chaque processus Unix hérite des fds ouverts de son parent. Pour un pipe `ls | grep`, le processus `grep` doit lire depuis le pipe (fd 0), pas depuis le terminal. L'appel `dup2(pipe_read_end, STDIN_FILENO)` remplace `stdin` par le pipe avant `exec`. C'est l'une des opérations les plus délicates de l'implémentation.

2.4 Mode raw terminal et édition de ligne

La lecture de ligne interactive — avec navigation par flèches, autocomplétion, `Ctrl+R` — nécessite de prendre le contrôle total du terminal. RustShell utilise `tcgetattr/tcsetattr` pour désactiver le mode canonique (buffering ligne par ligne) et l'écho automatique des caractères. En mode raw, chaque octet est traité individuellement :

Les séquences `ESC[A`, `ESC[B`, `ESC[C`, `ESC[D` correspondent aux flèches `↑↓←→`

`Ctrl+C` (code 3), `Ctrl+D` (4), `Ctrl+R` (18), `Ctrl+A` (1), `Ctrl+E` (5) sont interceptés

La fonction `redraw_line()` utilise les séquences ANSI `\r\x1b[nC\x1b[K` pour redessiner la ligne sans clignotement

Le terminal est toujours restauré à son état initial, même en cas d'interruption — garantissant que le terminal reste utilisable après une sortie brutale.

3. Fonctionnalités implémentées

3.1 Récapitulatif des cinq parties

Partie	Titre	Statut	Lignes	Tests
1	Lexer & Parser de commandes	Complet	~550	15
2	Exécuteur de commandes simples	Complet	~400	8
3	Pipes et redirections	Complet	~250	5
4	Gestion des jobs et signaux	Complet	~290	5
5	Historique et autocomplétion	Complet	~290	7
-	Prompt dynamique Git	Bonus	~130	4
-	Globbering * ? [...]	Bonus	~180	9
-	Substitution de commande \$()	Bonus	~60	—
-	Ctrl+R recherche historique	Bonus	~60	—

3.2 Fonctionnalités avancées (bonus)

- **Prompt dynamique**

Le prompt affiche en temps réel l'utilisateur, l'hôte, le répertoire courant avec raccourci ~, la branche Git active (lue directement depuis .git/HEAD sans lancer git), l'heure, et l'indicateur \$ coloré en rouge si la dernière commande a échoué. Ce comportement est identique à celui de shells professionnels comme zsh avec le thème Powerlevel10k.

- **Expansion glob**

L'algorithme de pattern matching est implémenté de zéro, sans crate externe. Il supporte l'étoile * (zéro ou plusieurs caractères), le point d'interrogation ? (exactement un caractère), les classes [abc], les plages [a-z], et la négation [!abc]. Les fichiers cachés (commençant par .) ne sont pas matchés sauf si le pattern commence explicitement par un point — comportement conforme à POSIX.

- **Substitution de commande**

La syntaxe \$(cmd) exécute une commande dans un sous-processus, capture sa sortie, et substitue le résultat dans la ligne courante. Exemple : echo "Fichiers : \$(ls src/*.rs | wc -l)" affiche le nombre de fichiers Rust du projet. Cette fonctionnalité est intégrée à l'expansion de variables et fonctionne de manière transparente dans tous les contextes.

- **Recherche inverse Ctrl+R**

En appuyant sur Ctrl+R en mode interactif, RustShell bascule dans un mode de recherche incrémentale dans l'historique. L'utilisateur tape un fragment et le shell affiche en temps réel la commande la plus récente correspondante. Entrée valide, Ctrl+C ou Ctrl+G annule.

4. Qualité du code et ingénierie

4.1 Métriques

Indicateur	Valeur
Lignes de code (hors commentaires)	~2 200 lignes effectives
Lignes totales (avec docs et tests)	2 929 lignes
Modules	8 modules avec responsabilités distinctes
Tests unitaires	49 tests — 0 échec — 0 test ignoré
Taille binaire release	531 Ko (LTO + strip activés)
Dépendances runtime	2 crates : libc + signal-hook
Dépendances absentes	Aucune : readline, ncurses, shell-words, glob
Profil release	opt-level=3, lto=true, codegen-units=1

4.2 Gestion des erreurs

Rust force le traitement explicite des erreurs via le type `Result<T, E>`. RustShell définit des types d'erreur sémantiques pour chaque couche :

`LexError::UnterminatedString` — chaîne entre guillemets non fermée

`LexError::UnexpectedChar(c)` — caractère non reconnu

`ParseError::EmptyCommand` — commande vide (ex: redirection seule)

`ParseError::UnexpectedToken` — syntaxe invalide

Les erreurs sont propagées jusqu'à la boucle principale qui les affiche en rouge et incrémente le code de retour (`$?` = 2). Le shell ne crashe jamais sur une entrée invalide — il affiche le message et propose un nouveau prompt.

4.3 Sécurité mémoire

L'ensemble du code unsafe est confiné à deux zones précisément délimitées : les appels `libc` pour les opérations système (`pipe`, `waitpid`, `tcgetattr`) et la gestion des descripteurs de fichiers bruts. Ces zones représentent moins de 5% du code total. Tout le reste — parsing, expansion, historique, `glob` — est en Rust safe, avec des garanties statiques contre les accès mémoire invalides.

Rust vs C pour ce type de projet

Un shell équivalent en C nécessiterait typiquement Valgrind pour détecter les fuites mémoire, AddressSanitizer pour les buffer overflows, et des centaines d'heures de debugging. En Rust, le compilateur rejette ces classes d'erreurs au moment de la compilation. Le temps économisé a été réinvesti dans les fonctionnalités et les tests.

5. Pertinence et cas d'adoption

5.1 Qu'est-ce qui rend RustShell adoptable ?

Un projet est adoptable lorsqu'il répond à un besoin réel, qu'il est suffisamment fiable pour être utilisé sans crainte, et qu'il est suffisamment ouvert pour être étendu. RustShell remplit ces trois conditions.

- **Utilisabilité réelle**

RustShell n'est pas un jouet académique : il peut remplacer bash pour des sessions de travail courantes. Les fonctionnalités essentielles sont présentes — pipes, redirections, variables, historique, autocomplétion, jobs — et les raccourcis clavier (Ctrl+R, Ctrl+A/E/K/U/W) correspondent aux attentes d'un développeur habitué à bash ou zsh.

- **Performance**

Le binaire release de 531 Ko démarre en moins d'une milliseconde — sans configuration, sans fichier de plugins, sans interpréteur Python. Pour des environnements contraints (conteneurs Docker, systèmes embarqués, environnements de CI minimal), cette légèreté est un avantage concret.

- **Extensibilité**

L'architecture modulaire permet d'ajouter des fonctionnalités sans toucher au cœur du shell. Ajouter un nouveau builtin revient à écrire une fonction et l'enregistrer dans `try_builtin()`. Ajouter un opérateur revient à étendre l'enum `Token` et les règles du parser. La structure invite à la contribution.

5.2 Scénarios d'usage concrets

Contexte	Avantage de RustShell
Éducation	Base de code claire et documentée pour enseigner fork/exec, pipes, signaux. Idéal pour les cours OS/Système.
Environnements conteneurisés	Binaire unique de 531 Ko, zéro dépendance runtime. Image Docker minimale possible.
Systèmes embarqués Linux	Faible empreinte mémoire, démarrage instantané, compilable pour ARM/RISC-V.
Shell de rescue/debug	Autonome même si bash/sh sont absents ou corrompus sur un système.
Base pour shell personnalisé	Architecture claire à étendre : ajouter des builtins métier, un DSL propriétaire, des hooks d'audit.

5.3 Feuille de route : vers un shell production-ready

RustShell v0.2 est une base solide. Les prochaines versions pourraient intégrer :

Globbering récursif `**` (style zsh/bash 4.0) et expansion de tilde `~user`
Structures de contrôle : `if/then/else`, `while`, `for` — pour l'exécution de scripts

Alias et fonctions shell : `alias ll='ls -la', fn() { ... }`
Arithmétique `$((expr))` et substitution de processus `<(cmd)`
Fichier de configuration `~/.rustshellrc` et prompt configurable via `$PS1`
Support macOS (adaptations terminos BSD) pour portabilité cross-platform

À terme, RustShell pourrait s'inscrire dans la mouvance des outils Unix réécrits en Rust (ripgrep, fd, exa, bat, zoxide). Ces projets ont démontré qu'une réécriture dans un langage moderne peut apporter des gains de performance et de sécurité significatifs tout en restant parfaitement compatibles avec l'écosystème existant.

6. Apprentissages et compétences développées

6.1 Programmation système

Ce projet a nécessité la maîtrise pratique d'un ensemble de concepts système avancés, généralement enseignés théoriquement mais rarement mis en œuvre :

Concept	Mise en pratique dans RustShell
Modèle de processus Unix	fork/execve pour chaque commande externe ; wait/waitpid pour la récupération des codes de retour
Descripteurs de fichiers	dup2() pour rediriger stdin/stdout/stderr avant exec ; gestion stricte de la fermeture des fds
Pipes Unix	libc::pipe() pour créer des canaux de communication inter-processus ; orchestration de pipelines N-commandes
Signaux POSIX	SIGINT/SIGTSTP ignorés dans le shell parent, restaurés dans les enfants ; SIGCHLD pour détecter les fins de jobs
Mode raw terminal	tcgetattr/tcsetattr pour contrôle fin du terminal ; séquences ANSI pour l'interface utilisateur
Environnement d'exécution	Variables d'environnement propagées aux enfants via env::set_var ; gestion de PATH, PWD, HOME, OLDPWD

6.2 Théorie des compilateurs appliquée

L'implémentation du lexer et du parser constitue une application directe de la théorie des automates et des langages formels. Le lexer est un automate à états finis qui reconnaît les tokens. Le parser est un analyseur syntaxique descendant récursif qui construit un arbre de syntaxe abstraite (AST). Ces techniques, habituellement réservées à la construction de compilateurs, s'avèrent universellement applicables à tout système qui interprète du texte structuré.

6.3 Rust avancé

Le projet a été l'occasion de pratiquer des aspects avancés du langage Rust : le système de types avec enums algébriques (Token, ParseError, JobStatus), le pattern matching exhaustif, les lifetimes implicites dans les iterators, l'interface FFI avec libc via le bloc unsafe, et la gestion d'erreurs sans exceptions via Result et le propagateur ?.

7. Conclusion

RustShell démontre qu'il est possible, dans un cadre académique et avec un langage appris récemment, de produire un logiciel système fonctionnel, testé, et documenté. En attaquant l'un des programmes les plus fondamentaux de l'informatique Unix — le shell — ce projet couvre un spectre exceptionnellement large de compétences : compilation, système, terminal, algorithmique, et génie logiciel.

Au-delà de l'exercice pédagogique, RustShell est un projet qui peut être présenté à un recruteur, déployé dans un environnement réel, et étendu vers un outil professionnel. Il illustre l'état de l'art de la programmation système en 2025 : un code safe par construction, performant, autonome, et maintenable.

La maîtrise de Rust et de la programmation système POSIX représente aujourd'hui l'une des compétences les plus valorisées dans l'industrie du logiciel, notamment dans les domaines de l'infrastructure, de la sécurité, du cloud, et des systèmes embarqués. Ce projet est une démonstration concrète de cette maîtrise.

Annexe — Commandes de référence

- **Installation et compilation**

```
git clone https://github.com/votre-compte/rustshell.git
cd rustshell
cargo build --release      # binaire dans target/release/rustshell
cargo test                 # lancer les 49 tests unitaires
bash demo.sh               # script de démonstration guidée
```

- **Référence des opérateurs**

Syntaxe	Effet
cmd1 cmd2	Pipe : stdout de cmd1 → stdin de cmd2
cmd > fichier	Redirection stdout (troncature)
cmd >> fichier	Redirection stdout (append)
cmd < fichier	Redirection stdin depuis fichier
cmd 2> fichier	Redirection stderr
cmd &> fichier	Redirection stdout + stderr
cmd &	Exécution en arrière-plan
cmd1 ; cmd2	Séquence inconditionnelle
cmd1 && cmd2	cmd2 exécutée si cmd1 retourne 0
cmd1 cmd2	cmd2 exécutée si cmd1 retourne non-0
\$(cmd)	Substitution de commande
.rs / ?.toml / [a-z]	Expansion glob

- **Ressources**

The Linux Programming Interface — Michael Kerrisk (référence système Linux)
POSIX.1-2017 — IEEE/The Open Group (spécification officielle)
The Rustonomicon — Rust unsafe programming guide
Writing a Unix Shell — Indradhanush Gupta (série tutoriels)
man 2 fork, man 2 execve, man 2 pipe, man 2 waitpid, man 3 tcgetattr